

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	K. Ganeswar Kumar	Reg. No.:	192572105
Section / Batch:	BE - CSE(CAI)	Date:	04/09/2026

Code of Conduct

I K. Ganeswar Kumar / 192572105 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Ganeswar Kumar

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

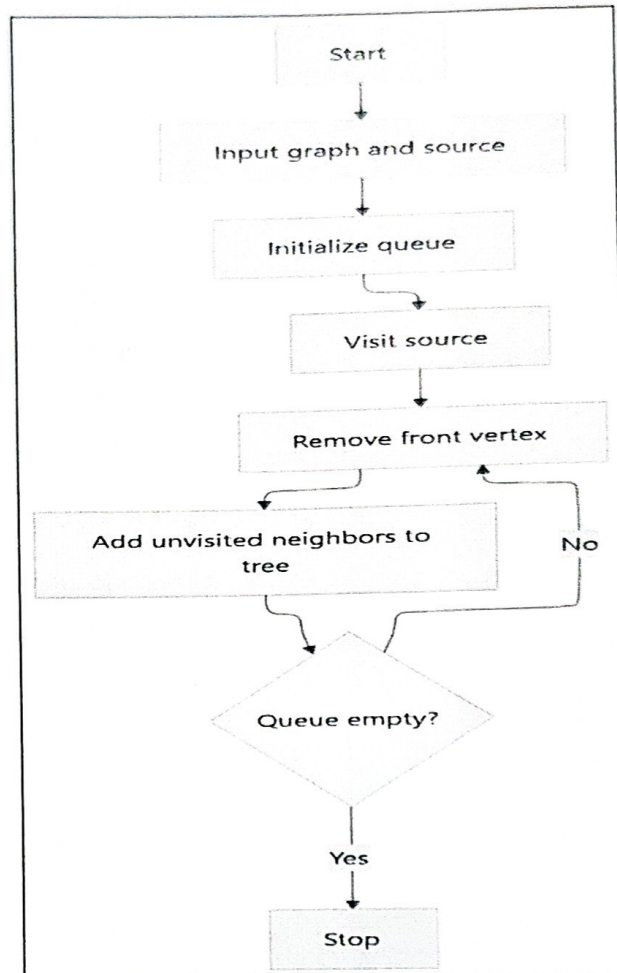
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

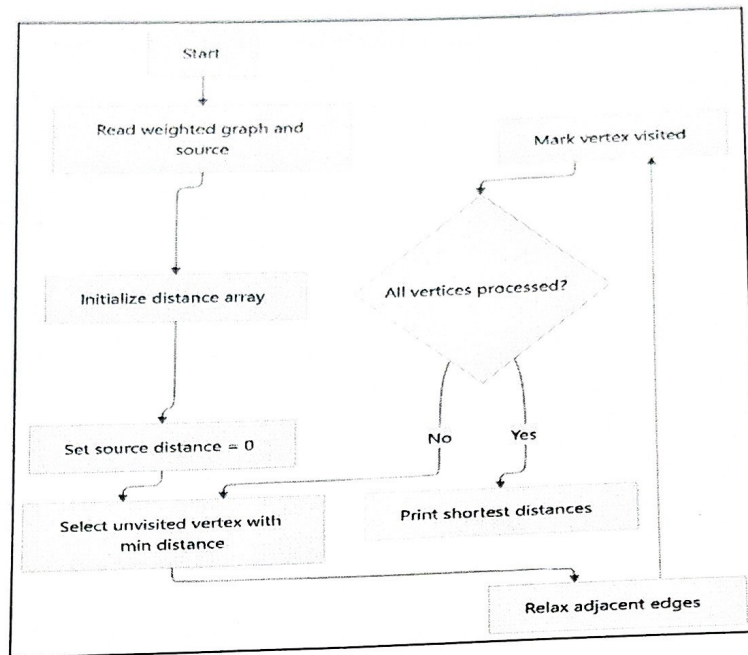
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

(20 Marks)

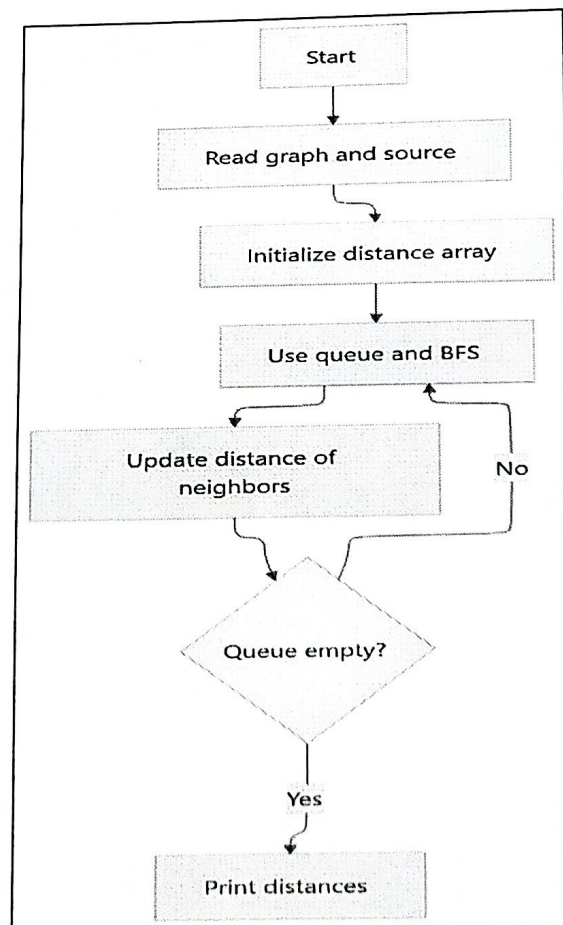
Flowchart Diagram:



3. Convert the flowchart for shortest path in an unweighted graph into code.

(20 Marks)

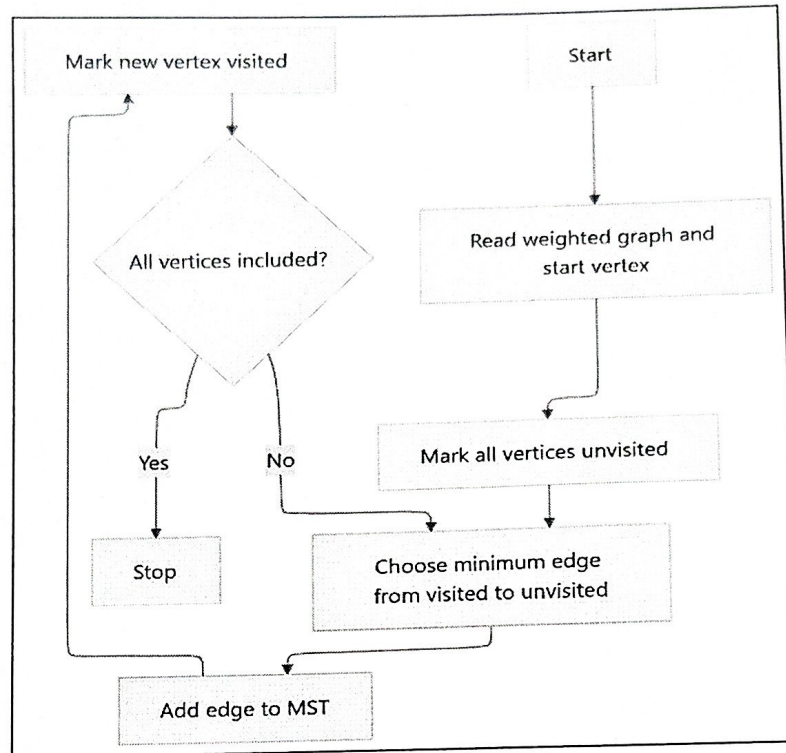
Flowchart Diagram:



4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

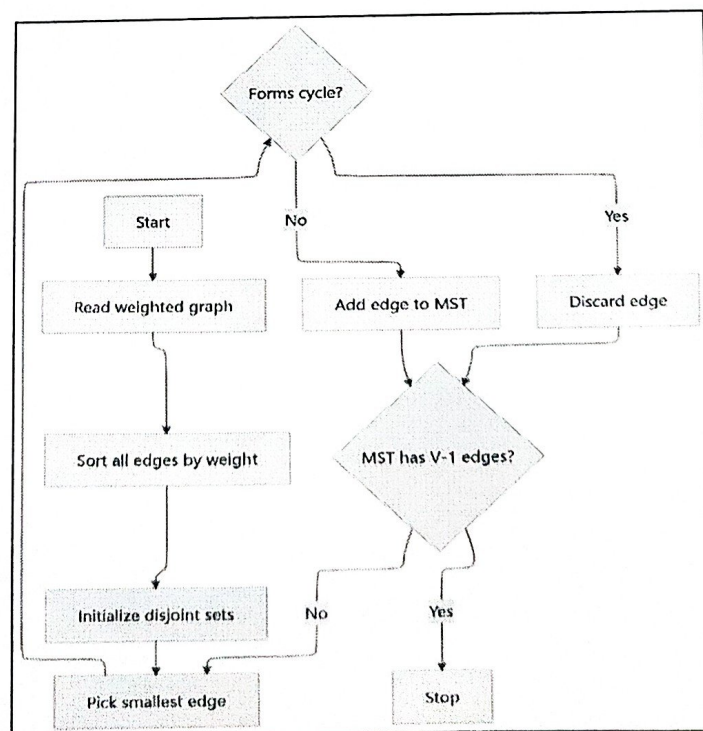
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:



TREE

```
#include <bits/stdc++.h>
```

```
#define MAX 20
```

```
int graph[MAX][MAX];
```

```
int visited[MAX];
```

```
int queue[MAX];
```

```
int front = 0, rear = 0;
```

```
int n;
```

```
void BFS (int start)
```

```
{
```

```
    visited[start] = 1;
```

```
    queue[rear++] = start;
```

```
    printf("BFS Traversal:");
```

```
    while (front < rear) :
```

```
    {
```

```
        int current = queue[front++];
```

```
        printf(" %d", current);
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            if (graph[current][i] == 1 && visited[i] == 0)
```

```
            {
```

```
                visited[i] = 1;
```

```
                queue[rear++] = i;
```

```
                printf("\nBFS Traversal Edge: %d -> %d",  
                    current, i);
```

```

}
}
}
Print ("In");
}
int main()
{
    int n;
    Print f("Enter number of vertices");
    scanf ("%d", &n);
    Print f("Enter adjacency matrix: \n");
    for (int i=0; i<n; i++)
    {
        for (int j=0; j<n; j++)
        {
            scanf ("%d", &graph[i][j]);
        }
    }
    Print f("Enter starting vertex:");
    scanf ("%d", &start);
    BFS(start);
    return 0;
}

```

*Output: Enter adjacency matrix:

```

0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

```

Enter Starting vertex: 0

BFS Traversal: 0

BFS Tree Edge: 0-1

BFS Tree Edge: 0-2

BFS Tree Edge: 1-2

SLA's shortest path
include <stdio.h>
include <stdlib.h>
#define MAX 100
int

Libra's Shortest path Algorithm

```

#include <stdio.h>
#include <limits.h>
#define MAX 20
int graph[MAX][MAX];
int distance[MAX];
int visited[MAX];
int n;

int find_min_vertex()
{
    int min = INT_MAX;
    int min_vertex = -1;
    for (int i = 0; i < n; i++)
    {
        if (visited[i] == 0 && distance[i] < min)
        {
            min = distance[i];
            min_vertex = i;
        }
    }
    return min_vertex;
}

void dijkstra's(int source)
{
    for (int i = 0; i < n; i++)
    {
        distance[i] = INT_MAX;
        visited[i] = 0;
    }
    distance[source] = 0;
}

```



```
For (int count=0; count<n-1; count++)
```

```
{
```

```
    int u = FindMinVertex();
```

```
    if (u == -1)
```

```
        break;
```

```
    visited[u] = 1;
```

```
    for (int v=0; v<n; v++)
```

```
    {
```

```
        if (visited[v] == 0 &&
```

```
            graph[u][v] != 0 &&
```

```
            distance[u] != INT_MAX &&
```

```
            distance[u] + graph[u][v] < distance[v])
```

```
        {
```

```
            distance[v] =
```

```
                distance[u] + graph[u][v];
```

```
        }
```

```
    }
```

```
}
```

```
    printf("Shortest distance from vertex %d: %d\n", source);
```

```
    for (int i=0; i<n; i++)
```

```
        printf("Enter source vertex: ");
```

```
        scanf("%d", &source);
```

```
        display(source);
```

```
    return 0;
```

```
}
```

* Output: Shortest distance from vertex:

vertex 0: 0

vertex 1: 10

vertex 2: 50

vertex 3: 30

vertex 4: 60

weighted Graph:

```
#include <stdio.h>
#define MAX 20
int graph (MAX)(MAX);
int visited (MAX);
int distance (MAX);
int parent (MAX);
int queue (MAX);
int n;
int front = 0;
int rear = 0;
void BFS (int source)
{
    for (int i = 0; i < n; i++)
    {
        visited [i] = 0;
        distance [i] = -1;
        parent [i] = -1;
    }
    visited [source] = 1;
    distance [source] = 0;
    queue [rear++] = source;
    while (front < rear)
    {
        int current = queue [front++];
        for (int i = 0; i < n; i++)
        {
            if (visited [i] == 0)
            {
                distance [i] = distance [current] + 1;
                parent [i] = current;
                queue [rear++] = i;
            }
        }
    }
}
```

```

    }
    }
}

void printPath (int vertex)
{
    if (vertex == -1)
        return;
    printPath (parent (vertex));
    printf ("%d", vertex);
}

int main ()
{
    int source;
    int destination;

    printf ("Enter number of vertices:");
    scanf ("%d", &n);
    printf ("Enter adjacency matrix:\n");
    for (int i=0; i<n; i++)
    {
        for (int j=0; j<n; j++)
        {
            scanf ("%d", &graph[i][j]);
        }
    }

    printf ("Shortest path:");
    printPath (destination);
    printf ("\n");
    return 0;
}

```

* Output:

Shortest distance = 3

Shortest path = 0 1 3 4

is Algorithm

```
#include <stdio.h>
#include <limits.h>
#define MAX 20
int graph[MAX][MAX];
int selected[MAX];
int n;
void print()
{
    int edgeCount = 0;
    int totalWeight = 0;
    selected[0] = 1;
    printf("\n Edges in minimum spanning Tree: \n");
    while (edgeCount < n-1)
    {
        int min = INT_MAX;
        int x = -1;
        int y = -1;
        for (int i = 0; i < n; i++)
        {
            if (selected[i] == 1)
            {
                for (int j = 0; j < n; j++)
                {
                    if (selected[j] == 0 &&
                        graph[i][j] != 0 &&
                        graph[i][j] < min)
                    {
                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }
    }
}
```


char's
#include
#d

```

    }
    }
    }
}

if (x == -1 || y == -1)
{
    Print ("graph is disconnected In");
    return;
}

Print ("%d = %d : %d\n", x, y, min);
Global weight += min;
selected[y] = 1;
edge count++;
}

Print ("In Total MST weight = %d\n", Global weight);
}

int main ()
{
    Print ("Enter number of vertices");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf ("%d", &graph[i][j]);
        }
    }
    Print();
    return 0;
}

```

* Output: Enter weighted adjacency matrix Edges in minimum spanning tree

0 2 0 6 0
2 0 7 8 5
0 3 0 0 7
6 9 0 0 9
0 5 7 9 0

0-1 = 2
1-3 = 3
1-4 = 5
0-3 = 6

Total MST weight = 16

Kruskal's Algorithm:-

```
#include <stdio.h>
```

```
#define MAX 100
```

```
typedef struct
```

```
{
```

```
    int source;
```

```
    int destination;
```

```
    int weight;
```

```
} Edge;
```

```
Edge edges[MAX];
```

```
int parent[MAX];
```

```
int n;
```

```
int edge count;
```

```
int find (int vertex)
```

```
{
    if (parent[vertex] == vertex)
```

```
        return vertex;
```

```
    return parent[vertex] = find (parent[vertex]);
}
```

```
{
```

```
    void union set (int a, int b)
```

```
{
    int root A = find (a);
```

```
    int root B = find (b);
```

```
    parent [root B] = root A;
```

```
}
```

```
void sort Edges()
```

```
{
    for (int i = 0; i < edge count - 1; i++)
```

```
    {
        for (int j = 0; j < edge count - i - 1; j++)
```

```
        {
            if (edges[j].weight > edges[j+1].weight)
```

```
            {
```

```

    }
}
}
void kruskal()
{
    int n = 6;
    int totalWeight = 0;
    sort(edges);
    for (int i = 0; i < n; i++)
    {
        parent[i] = i;
    }
    Print("\n Edges in Minimum spanning Tree:\n");
    for (int i = 0; i < edges.count() & n < n-1; i++)
    {
        int u = edges[i].source;
        int v = edges[i].destination;
        int rootU = find(u);
        int rootV = find(v);
        if (rootU != rootV)
        {
            Print("%d - %d : %d\n", u, v, edges[i].weight);
        }
        kruskal();
        return 0;
    }
}

```

* Output: Enter edges: Edges in Minimum spanning Tree:

0 1 2
 0 3 6
 1 2 3
 1 3 8
 1 4 5
 2 4 7
 3 4 9

0-1 : 2
 1-2 : 3
 1-4 : 5
 0-3 : 6

Total MST weight = 11

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: <u>U. P. J.</u>	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____